C语言自定义类型详解-联合体&枚举类型

深入剖析联合体内存共用原理与枚举常量应用

1. 联合体

1.1 联合体的声明

联合体也是由一个或多个成员构成，这些成员可以是不同的类型，但是编译器只为最大的成员分配足够的内存空间。

联合体的特点是所有成员共用同一块内存空间，所以联合体也叫：**共用体**。

```
union Un
{
    char c;
    int i;
};

int main()
{
    //联合变量的定义
    union Un un = { 0 };
    //计算联合变量的大小
    printf("%d\n", sizeof(un));
    return 0;
}
```

```
输出结果为：4
```

因为联合的成员是共用同一块内存空间的，这样一个联合变量的大小，至少是最大成员的大小。但是不一定大小正好是这个最大成员的字节大小。

1.2 联合体的特点

联合的成员用同一块内存空间的，这样一个联合变量的大小，至少是最大成员的大小（因为联合至少得有能力保存最大的那个成员）。

```
//代码1
#include "stdio.h"
//联合类型的声明
union Un
{
    char c;
    int i;
};

int main()
{
    //联合变量的定义
    union Un un = 0[];
    //打印两个成员的地址，看看吗？
    printf("%p\n", &(un.i));
    printf("%p\n", &(un.c));
    printf("%d\n", sizeof(un));
    return 0;
}
```

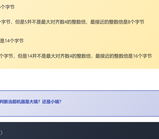
```
//代码2
#include "stdio.h"
//联合类型的声明
union Un
{
    char c;
    int i;
};

int main()
{
    //联合变量的定义
    union Un un = 0[];
    un.i = 0x11223344;
    un.c = 0x55;
    printf("%p\n", un.i);
    return 0;
}
```

输出的结果：

```
004AFB5C
004AFB5C
004AFB5C
```

```
11223355
```

代码1输出的三个地址一模一样，代码2的输出，我们发现两种的第4个字节的内容修改为了7，我们对输出分析就可以搞出，un的内存布局图。



1.3 相同成员的结构体和联合体对比

我们两对比一下相同成员的结构体和联合体的内存布局情况。

```
struct S
{
    char c;
    int i;
};
struct S s = 0[];
```

```
union Un
{
    char c;
    int i;
};
union Un un = 0[];
```

结构体和联合体的内存对比

1.4 联合体大小的计算

联合体大小计算规则

- 联合体大小至少是最大成员的大小。
- 当最大成员大小不是最大对齐数的整数倍的时候，就要对齐到最大对齐数的整数倍。

```
union Un1
{
    char c[5];
    int i;
};

union Un2
{
    short c[7];
    int i;
};

int main()
{
    //下面输出的结果是什么？
    printf("%d\n", sizeof(union Un1));
    printf("%d\n", sizeof(union Un2));
    return 0;
}
```

第一个联合体：
char类型数组为小于等于
int型4个字节
理论上应该是5个字节，但是5并不是最大对齐数4的整数倍，最接近的整数倍是8个字节

第二个联合体：
short类型的数组是14个字节
int类型4个字节
理论上应该是14个字节，但是14并不是最大对齐数4的整数倍，最接近的整数倍是16个字节

1.5 练习

写一个程序，判断当前机器是大端？还是小端？

```
int check_sys()
{
    union
    {
        int i;
        char c;
    }un;
    un.i = 1;
    return un.c;//返回对是小端，返回的是大端
}
```

2. 枚举类型

2.1 枚举的声明及特点

都可以使用枚举进行表示。相反一些不是有限集合的就不可以用枚举来表示。

例如，不适合使用枚举的情况：

❌ 年龄的数值
❌ 温度
❌ 相对上所有人的名字

举例：

```
enum Day //星期
{
    Mon,
    Tues,
    Wed,
    Thur,
    Fri,
    Sat,
    Sun
};

enum Color //颜色
{
    RED,
    GREEN,
    BLUE
};
```

小结：enum Day，enum Color是枚举类型。一中的内容是枚举类型的可能取值，也叫枚举常量。

```
enum Day //星期
{
    Mon,
    Tues,
    Wed,
    Thur,
    Fri,
    Sat,
    Sun
};
int main()
{
    //下面输出的结果是什么？
    printf("%d\n", Mon);
    printf("%d\n", Tues);
    printf("%d\n", Wed);
    return 0;
}
```

小结：默认值是从0开始，依次递增1，也可以赋初值。

```
enum Color
{
    RED = 2,
    GREEN = 4,
    BLUE = 8
};
int main()
{
    //下面输出的结果是什么？
    printf("%d\n", RED);
    printf("%d\n", GREEN);
    printf("%d\n", BLUE);
    return 0;
}
```

2.2 枚举类型的使用

```
enum Color //颜色
{
    RED = 2,
    GREEN = 4,
    BLUE = 8
};
int main()
{
    enum Color clr = GREEN;
    printf("%d\n", clr);
    return 0;
}
```

练习：模拟交通灯时状态转换（红→绿→黄→红），使用枚举表示三种状态。

```
#include "stdio.h"

//定义交通灯状态的枚举
enum TrafficLight
{
    RED,    // 默认值0
    GREEN,  // 默认值1
    YELLOW  // 默认值2
};

int main()
{
    enum TrafficLight light = RED;  // 初始状态为红灯

    // 模拟状态变化
    for (int i = 0; i < 3; i++)
    {
        switch(light)
        {
            case RED:
                printf("红灯亮\n");
                light = GREEN;  // 红灯→绿灯
                break;
            case GREEN:
                printf("绿灯亮\n");
                light = YELLOW  // 绿灯→黄灯
                break;
            case YELLOW:
                printf("黄灯减速\n");
                light = RED;    // 黄灯→红灯
                break;
        }
    }
    return 0;
}
```

📋 本节总结

- 联合体（union）所有成员共用同一块内存。
- 联合体的大小至少是其最大成员的大小，且需要对齐到最大对齐数的整数倍。
- 联合体用来节约内存和优化数据结构大小场。
- 枚举（enum）用于定义常量集合，增强代码可读性。
- 枚举常量默认从0开始递增，也可以手动指定初始值。
- 枚举非法优势（类型安全）：等值语句中非常实用。